LuxStudio

Technical Requirements Document

Flutter Android Application Architecture, Technology Stack and Implementation Requirements

Document Status: Technical Planning / Development Specification

Primary Platform: Android

Main Framework: Flutter

Prepared: 23 August 2026

1. Technical Overview

LuxStudio is an Android-first mobile application designed to help users turn long-form videos into polished short-form content. The application will support importing media, editing clips, detecting and removing silence, generating captions, applying branding, using AI to suggest short clips, generating social-media copy and exporting vertical videos.

Because video processing and AI analysis can be computationally intensive, the recommended solution is a hybrid architecture: Flutter will provide the mobile user interface and application logic, while heavier AI and video-processing tasks can be handled through a backend service or specialised native/cloud processing components.

2. Technical Goals

• Build a modern Android application using Flutter.

• Provide a responsive and simple user experience for video editing.

• Keep the application modular so new features can be added later.

• Protect user data, media files and API credentials.

• Support both local device operations and cloud-based processing where required.

• Avoid blocking the interface while long-running video or AI tasks are executing.

• Create a technical foundation that can later support iOS without rebuilding the entire application.

3. Recommended High-Level Architecture

Presentation Layer: Flutter screens, widgets and UI components.

Application Layer: State management, business logic, validation and workflow coordination.

Data Layer: Local storage, repositories, API clients and media metadata.

Processing Layer: Video processing, transcription, caption timing and export operations.

Backend / AI Layer: Authentication, cloud processing, AI analysis, clip generation and job management.

Suggested architecture pattern: **Clean Architecture + Feature-Based Folder Structure**.

Each major feature should be separated into presentation, domain and data responsibilities where practical. This will make the codebase easier to test and maintain as LuxStudio grows.

4. Primary Technology Stack

Technology Area	Recommended Technology	Purpose
Mobile Framework	Flutter + Dart	Cross-platform application development with Android as the first ta
State Management	Riverpod	Manage application, editing and asynchronous processing state.
Navigation	GoRouter	Structured navigation and route management.
Networking	Dio	Communicate with backend APIs and manage HTTP requests.
Backend API	FastAPI (Python)	Provide secure APIs and coordinate AI/video-processing jobs.
Database	PostgreSQL	Store users, projects, settings, job metadata and project records.
Authentication	Firebase Authentication	User sign-in and account identity management.
Cloud Storage	Firebase Storage or object storage	Store uploaded source media and generated files.
Local Database	Isar or SQLite	Cache projects, settings and local metadata.
Video Playback	Flutter video player package / native integration	Preview imported and edited video.
Video Processing	FFmpeg-based service or native integration	Trim, concatenate, analyse audio and export video.
Speech-to-Text	Whisper-family model/service	Generate transcripts and timed captions.
AI Analysis	LLM service through backend	Identify useful clips and generate summaries and social copy.
Background Jobs	Backend job queue / worker system	Run long video and AI operations outside the mobile request cycle
Crash Monitoring	Firebase Crashlytics	Capture production crashes and errors.
Analytics	Firebase Analytics	Measure feature usage and application behaviour.
Version Control	Git + GitHub	Source control, collaboration and project tracking.

5. Flutter Application Requirements

The Flutter application will be responsible for the main mobile experience. It should not contain long-running server API keys or directly expose sensitive AI credentials.

5.1 Core Flutter Features

• Android application targeting current supported Android versions.

• Responsive layouts for different Android screen sizes.

• Material Design 3-based interface with a custom LuxStudio design system.

• Dark and light mode support as a recommended enhancement.

• Reusable widgets for buttons, cards, progress states, media previews and error messages.

• Centralised theme, spacing and typography configuration.

• Accessible text sizing and touch targets.

5.2 Recommended Flutter Packages

Package / Category	Use
flutter_riverpod	Application and feature state management.
go_router	Navigation and route handling.
dio	REST API communication, interceptors and request handling.
freezed / json_serializable	Immutable models and JSON model generation.
video_player	Video playback and preview.
file_picker / image_picker	Select video, audio and image files.
permission_handler	Request and manage Android permissions.
firebase_core	Firebase application integration.
firebase_auth	User authentication.
firebase_storage	Cloud media storage when Firebase Storage is selected.
firebase_crashlytics	Crash and error reporting.
firebase_analytics	Application usage analytics.
flutter_secure_storage	Securely store tokens and sensitive local values.
connectivity_plus	Monitor network availability.

6. Feature-Level Technical Requirements

6.1 Authentication and User Accounts

• Support secure sign-up and sign-in.

• Use Firebase Authentication or an equivalent identity provider.

• Store authentication tokens securely on the device.

• Provide logout and session-expiry handling.

• Associate projects and cloud processing jobs with an authenticated user.

6.2 Project Management

• Create, rename, save, reopen and delete projects.

• Store project metadata locally and optionally synchronise it with the backend.

• Track source media, timeline state, captions, branding settings and generated outputs.

• Use unique project identifiers.

• Provide recovery for incomplete or interrupted operations where technically possible.

6.3 Media Import

• Allow users to select supported video and audio files.

• Validate file type and basic media information before processing.

• Display filename, duration, resolution and orientation when available.

• Use Android storage and media permissions only when required.

• Keep original source media unchanged.

6.4 Timeline and Basic Editing

• Represent editing operations as structured timeline data rather than immediately modifying the original video.

• Support trim, split, delete, reorder and merge operations.

• Maintain a project edit model that can later be converted into a final processing job.

• Support undo and redo using an edit-action history where feasible.

6.5 Silence Detection and Removal

• Send media or audio analysis jobs to the processing layer when local processing is insufficient.

• Allow configuration of minimum silence duration and detection threshold.

• Return detected time ranges to the Flutter application for user review.

• Store accepted removal ranges as project edit data.

• Ensure remaining clips remain synchronised.

6.6 Captions

• Create a transcript with timestamps.

• Store captions as editable structured data: text, start time, end time and style information.

• Allow correction of transcript errors.

• Support phrase-level or word-level timing where available.

• Render captions within safe areas for vertical video.

6.7 AI Clip Generation

• Send transcript and timing information to the backend AI service.

• Request multiple candidate clips rather than one fixed result.

• Return title/topic, start time, end time, duration and confidence/score where supported.

• Allow users to regenerate suggestions.

• Require user review before final export or publishing.

6.8 AI Summary and Social Copy

• Generate an editable title, summary, key points, description and hashtags.

• Do not treat AI output as guaranteed factual information.

• Allow users to copy generated text.

• Keep AI requests behind the backend so provider keys are not exposed in the APK.

6.9 Branding

• Store logo references, brand name and reusable style settings.

• Allow branding settings to be changed without releasing a new application version.

• Apply branding during preview or final rendering according to the selected project configuration.

6.10 Export

• Primary output should support 1080 × 1920 vertical MP4.

• Use a processing job for complex rendering and export.

• Show queued, processing, completed and failed states.

• Allow completed files to be saved or shared through Android-supported flows.

• Use meaningful output filenames.

7. Backend Requirements

A backend is recommended because AI clip analysis, transcription and video rendering may exceed the practical limits of many Android devices. The backend should be designed as a separate service so Flutter remains focused on the mobile experience.

Recommended backend stack

Component	Recommendation
API Framework	FastAPI with Python
API Style	REST API initially; asynchronous job endpoints for long tasks
Database	PostgreSQL
ORM / Data Access	SQLAlchemy or equivalent
Background Processing	Worker and queue architecture
Video Processing	FFmpeg in controlled server/container environment
AI Integration	Backend adapters for transcription and LLM providers
Storage	Cloud object storage or Firebase Storage
Deployment	Container-based deployment with environment variables and secrets

7.1 Asynchronous Processing Model

Long-running operations should not keep a normal HTTP request open until processing finishes. The recommended workflow is:

1. Flutter uploads media or sends a processing request.

2. The backend creates a processing job with a unique job ID.

3. The backend queues the job.

4. A worker performs transcription, silence analysis, AI analysis or video rendering.

5. The Flutter application checks job status or receives status updates.

6. The completed result is saved and linked to the project.

8. Data Model Requirements

Entity	Important Fields
User	id, authentication reference, display name, createdAt
Project	id, userId, name, status, sourceMediaId, createdAt, updatedAt
MediaAsset	id, projectId, storagePath, filename, duration, resolution, orientation
TimelineItem	id, projectId, mediaAssetId, startTime, endTime, position
Caption	id, projectId, text, startTime, endTime, styleId
CaptionTemplate	id, userId or systemId, font, size, colours, animation settings
ClipCandidate	id, projectId, title, startTime, endTime, duration, status
BrandSettings	id, userId/organisationId, logoPath, name, visual settings

Entity	Important Fields
ProcessingJob	id, projectId, type, status, progress, errorMessage, createdAt
GeneratedContent	id, projectId, title, summary, description, hashtags
Export	id, projectId, outputPath, resolution, status, createdAt

9. API Requirements

The backend should expose versioned endpoints, for example **/api/v1/...**. Requests must be authenticated where user-specific data is accessed.

Area	Example Operations
Authentication	Sign in, sign out, validate session
Projects	Create, list, read, update and delete projects
Media	Upload media, retrieve metadata, access processed output
Captions	Generate transcript, retrieve captions, update caption text
Silence	Create silence-analysis job and retrieve detected ranges
AI Clips	Generate candidate clips and retrieve results
AI Content	Generate title, summary, description and hashtags
Processing Jobs	Create job, check status, retrieve progress and errors
Export	Create export job and retrieve final output

10. Security Requirements

• Never place AI provider keys, database passwords or private backend credentials inside the Flutter source code or APK.

• Use HTTPS for all production API communication.

• Use authenticated and authorised API requests.

• Store mobile authentication tokens using secure device storage.

• Validate uploaded files and enforce supported file types and size limits.

• Use signed or controlled URLs for private media where applicable.

• Keep secrets in environment variables or a managed secret system.

• Log errors without exposing passwords, tokens or sensitive media information.

11. Performance Requirements

• Normal navigation and basic UI interactions should remain responsive.

• Long-running operations must show visible progress.

• The application should not freeze while a processing job is running.

• Large media uploads should support retry or recovery where practical.

• Video previews should use appropriately sized streams or files when possible.

• Processing results should be cached where repeated work can be avoided.

• The backend should be designed to scale processing workers independently from the API.

12. Testing Requirements

Testing Type	Requirement
Unit Testing	Test business logic, validation, repositories and state-management logic.
Widget Testing	Test important Flutter screens and reusable components.
Integration Testing	Test complete flows such as login, project creation and processing status.
API Testing	Validate authentication, project operations, job creation and error responses.
Device Testing	Test on multiple Android devices and screen sizes.
Media Testing	Test short and long videos, different resolutions and expected failure cases.
Security Testing	Check authentication, permissions, secret handling and API access control.
User Acceptance Testing	Confirm that the main workflow is understandable for target users.

13. Development Environment and Tools

• **IDE:** Android Studio and/or Visual Studio Code.

• **Flutter SDK:** Stable release channel.

• **Language:** Dart for the mobile application; Python for the recommended backend.

• **Android Development:** Android SDK, emulator and physical device testing.

• **Version Control:** Git with GitHub repository management.

• **API Documentation:** OpenAPI/Swagger generated from FastAPI.

• **Design:** Figma for UI design and component planning.

• **CI/CD:** GitHub Actions or equivalent automation for tests and builds.

14. Suggested Flutter Project Structure

lib/
core/ — shared configuration, theme, errors, utilities and network setup
features/ — feature-based modules such as auth, projects, editor, captions, clips and export
Each feature can contain **data/**, **domain/** and **presentation/** folders.
shared/ — reusable widgets and shared models
main.dart — application entry point

15. MVP Technical Priorities

Priority	Technical Deliverable
P0	Flutter project setup, theme, navigation and state-management foundation
P0	Authentication and secure session handling
P0	Project creation and media import

Priority	Technical Deliverable
P0	Video preview and structured basic editing model
P0	Backend processing job architecture
P0	Automatic transcription and editable captions
P0	Silence detection/removal workflow
P0	1080 × 1920 MP4 export workflow
P1	AI clip candidate generation
P1	AI summaries, descriptions and hashtags
P1	Branding configuration
P2	Batch processing
P2	Cloud collaboration
P2	Advanced analytics and clip scoring

16. Technical Definition of Done

• The Flutter Android application builds successfully in a production configuration.

• Users can authenticate and manage their own projects.

• Users can import supported media and create a project.

• Core editing data is saved without changing the original source media.

• Silence removal, captions and export workflows operate through reliable processing jobs.

• The application clearly communicates loading, progress, completion and failure states.

• AI provider credentials are never exposed in the mobile application.

• Important features are covered by appropriate automated and manual testing.

• The backend API and Flutter application are documented sufficiently for future development.

• The MVP can complete the core workflow from media import to vertical video export.

17. Final Recommended Stack Summary

Frontend: Flutter + Dart + Riverpod + GoRouter + Dio
Authentication: Firebase Authentication
Local Storage: Isar or SQLite
Cloud Storage: Firebase Storage or object storage
Backend: Python + FastAPI
Database: PostgreSQL
Video Processing: FFmpeg-based processing service
Speech-to-Text: Whisper-family transcription
AI: LLM integration through the backend
Monitoring: Firebase Crashlytics + Firebase Analytics
Version Control: Git + GitHub
CI/CD: GitHub Actions or equivalent

This technical requirements document is intended to guide the architecture and implementation of LuxStudio as a Flutter-based Android application. The technology choices may be refined during prototyping, especially for video

rendering and AI processing, but Flutter remains the primary mobile application framework.